BC7215A Offline Air Conditioner Control Library

ESP-IDF (C++/FreeRTOS) Demo

and

C++ Library

User Manual

- **Directly uses the idf flashing/debug terminal and can be used with any ESP32**
- **Demonstrates all library functions, including air conditioner control and IR command decoding**

Preface

This demo program is created based on ESP-IDF V5.5.4. It directly uses the flashing/debug terminal as the interactive operation interface. For the hardware connection, four I/O pins are needed and can be defined in code arbitrarily. When porting to a different ESP32 model, you only need to use idf.py set-target <chip model> to change the chip model and modify the corresponding I/O pin configuration lines in main.cpp.

In this example, the standard C-language BC7215A air conditioner control library is wrapped in C++ to form two classes: BC7215 and BC7215AC. The BC7215 class is the basic driver class for the BC7215 chip and can be used independently. BC7215AC is the air conditioner control class. It uses the BC7215 class as a member and simplifies the user interface. The code has also been adapted for the ESP-IDF FreeRTOS environment.

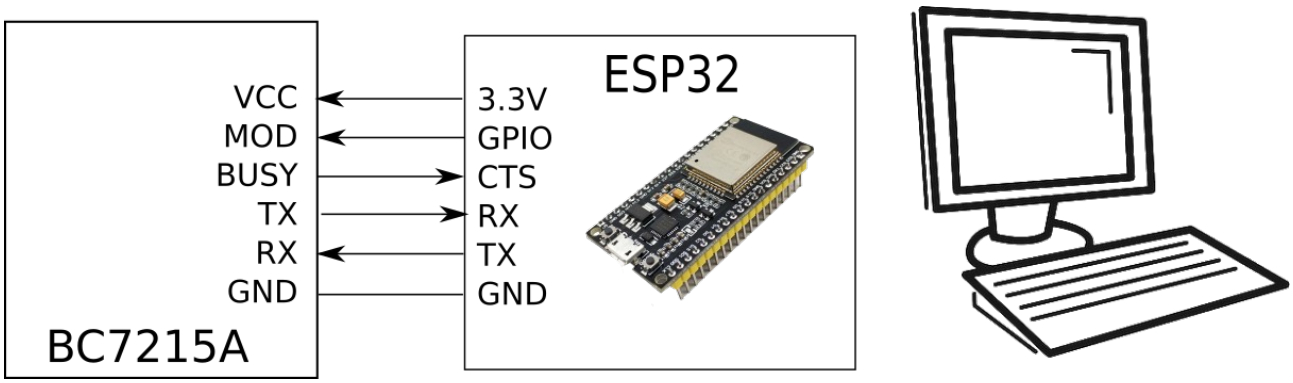
Other development environments that use C++ and FreeRTOS can also refer to this example.

Hardware Connection and Configuration

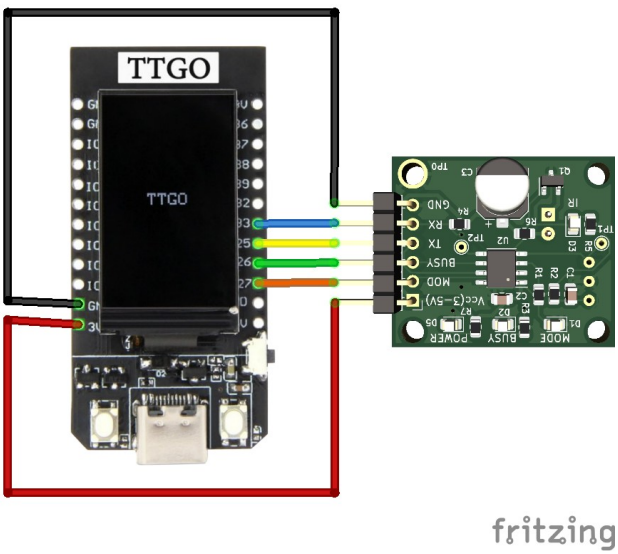
The demo requires the following I/O resources:

One UART with a CTS signal: used to connect to the BC7215A

GPIO x1: configured as output and connected to the MOD pin of the BC7215A



This example uses the same ESP32 TTGO T-Display module and wiring as the Arduino example. Users can easily change to their own module and wiring.



Directory Structure

```
Project_Name (project root directory)
├── main
│   ├── include
│   │   └── bc7215_lib_config.h   <-- BC7215 driver configuration file
│   │
│   ├── main.cpp                  <-- Main program
│   └── CMakeList.txt
│
├── components
│   ├── bc7215
│   │   ├── include
│   │   │   └── bc7215.hpp
│   │   │
│   │   ├── bc7215.cpp            <-- BC7215 driver C++ wrapper
│   │   └── CMakeLists.txt
│   │
│   ├── bc7215ac
│   │   ├── include
│   │   │   └── bc7215ac.hpp
│   │   │
│   │   ├── bc7215ac.cpp         <-- Air conditioner library C++ wrapper
│   │   └── CMakeLists.txt
│   │
│   └── bc7215_ac_lib
│       └── CMakeLists.txt
│
├── sdkconfig
└── CMakeList.txt

bc7215_ac_lib            <-- External, source files of the AC control library
├── bc7215_types.h
├── bc7215_lib.h
├── bc7215_ac_lib.h
├── bc7215_lib.c
├── bc7215_ac_lib.c
└── (other files)
```

Usage

Build and Flash

Important note: because FreeRTOS and UART hardware flow control are used, the contents of bc7215_lib_config.h differ from the template file.

After modifying the pin definitions in main.cpp according to your hardware connection, use it like a normal ESP-IDF project. First set the processor being used:

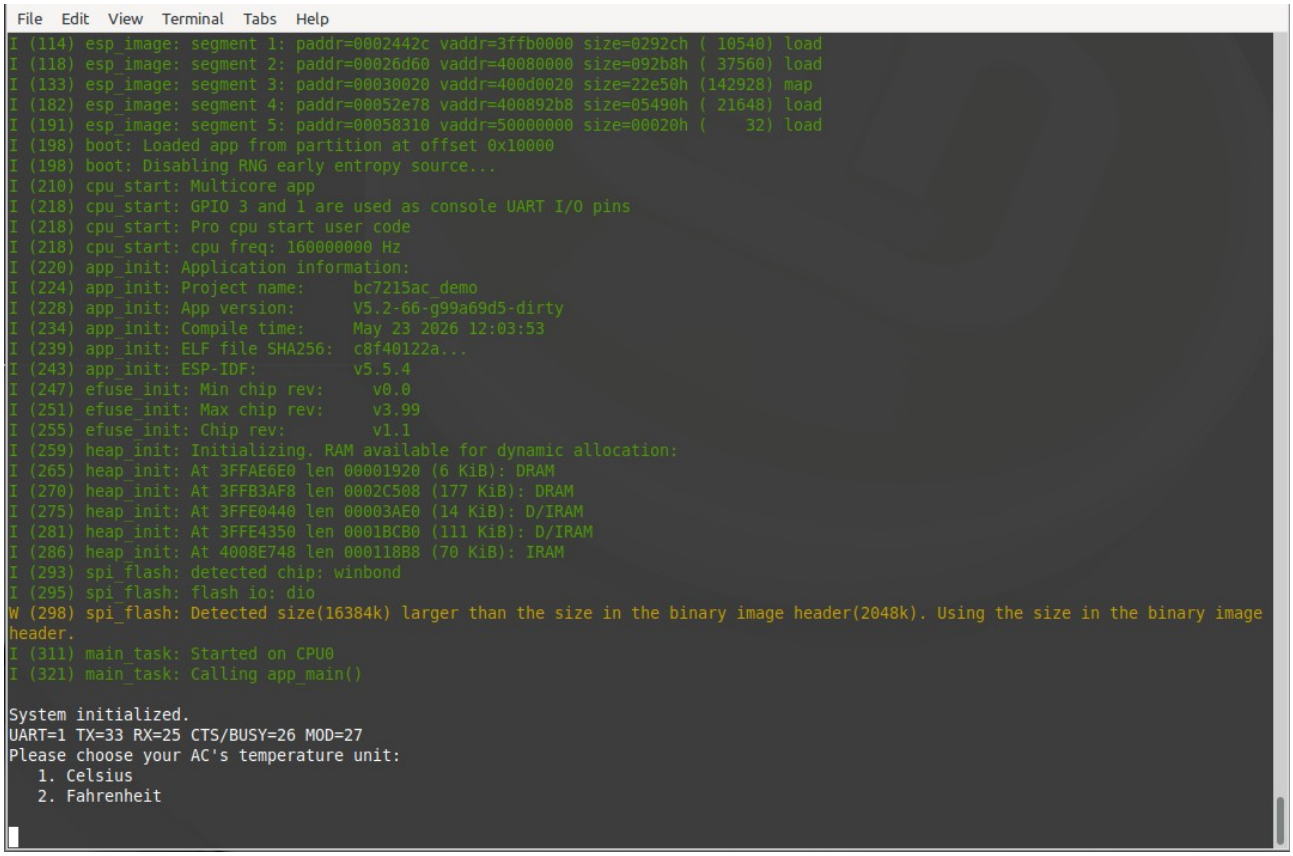
$> idf.py target <chip model> <== The chip model can be any ESP32-series chip

$> idf.py build <== Build the project

$> idf.py -p <serial port device> flash monitor <== Flash and enter the debug terminal

How to Use

After the example program starts, it first asks the user to select whether the controlled air conditioner uses Celsius or Fahrenheit, because air conditioners using different temperature units need to call different functions.

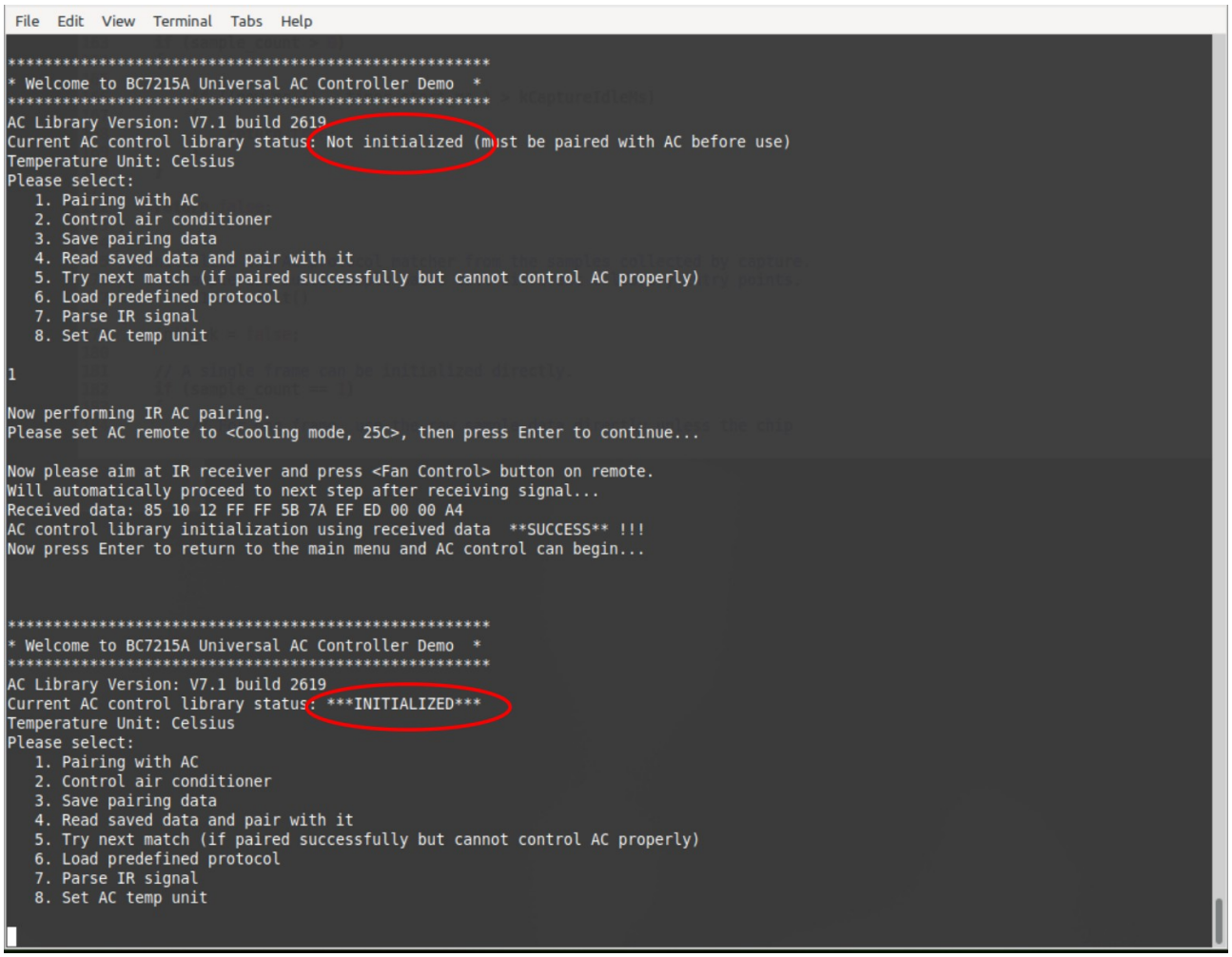


After that, the main menu is displayed in the terminal. The main menu shows the AC library version and the current status of the air conditioner code library, that is, whether it has been initialized (paired) with the air conditioner.

After the main menu is displayed, enter the menu number in the terminal and press Enter to enter the corresponding function.

1. Initialization (Pairing)

After the program starts, it should first complete pairing (initialization) with the air conditioner before any other functions can be operated. The operation process is basically the same as that of the DEMO software on the PC. Users can refer to the section about the DEMO software in the BC7215A DEMO board manual. In this example, IR signal capture and "initialization" are combined into one "pairing" operation.



After a menu operation is selected, the next operation prompt is displayed. The figure above shows the pairing process.

2. Air Conditioner Control

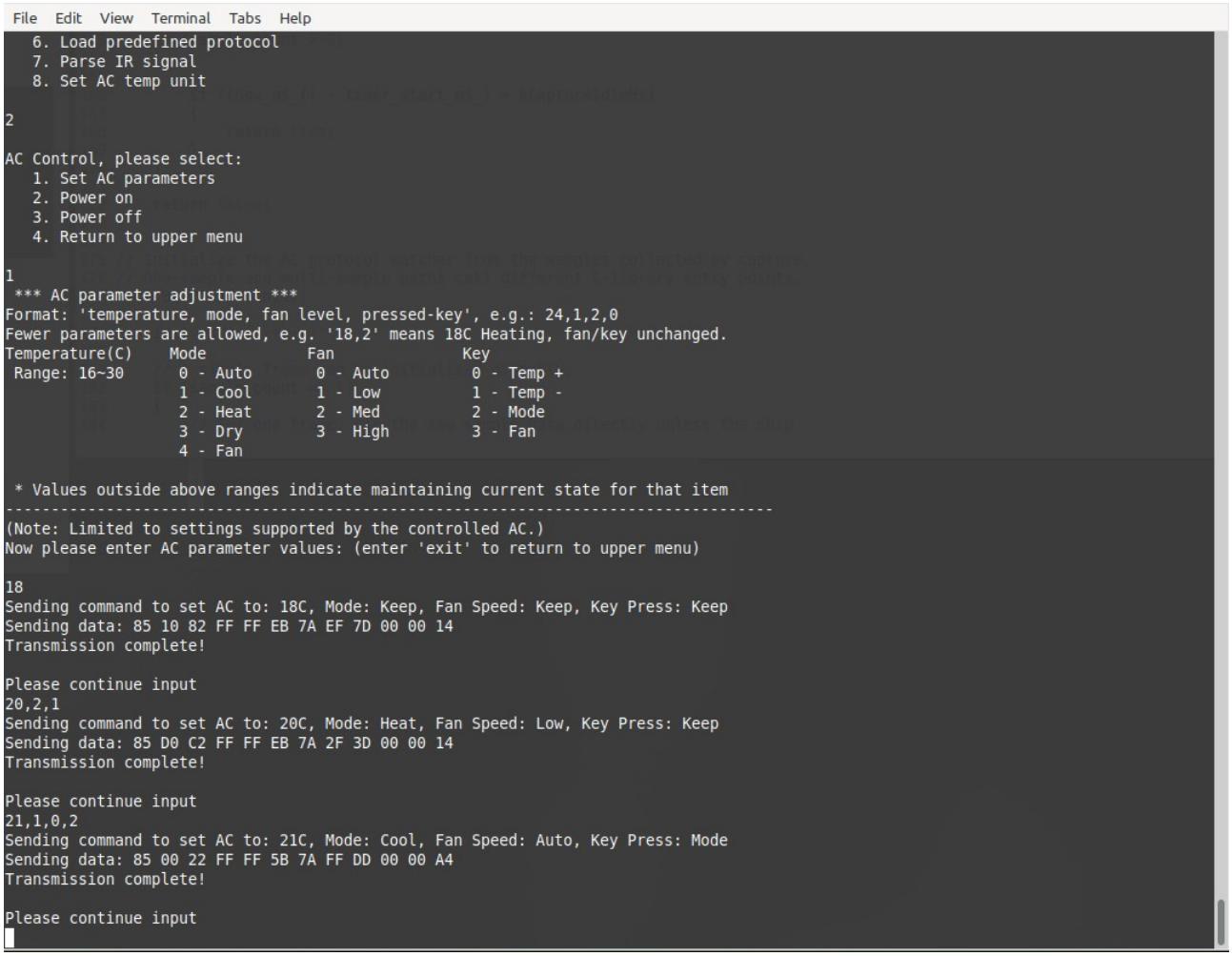
After initialization is completed, the air conditioner can be controlled. Select '2' in the main menu to enter the air conditioner control submenu.

The air conditioner control menu has four items. Select '1' to enter the air conditioner parameter control page, '2' or '3' to control power on/off, and '4' to return to the main menu.
On the parameter control page, enter four parameters in order: "temperature, mode, fan speed, remote-control key". Separate them with commas. Partial input is supported. For example, if you only need to change the temperature, you can enter only the temperature; in this case the other settings remain unchanged.

Enter the English word "exit" to exit air conditioner parameter control and return to the upper-level menu.

The valid input ranges are displayed when entering the parameter control page. Inputs outside the valid range are treated as keeping the corresponding setting unchanged. After a setting value is entered, the terminal displays the received setting, immediately starts IR transmission, and displays the transmitted data. After transmission is complete, it waits for the next input.

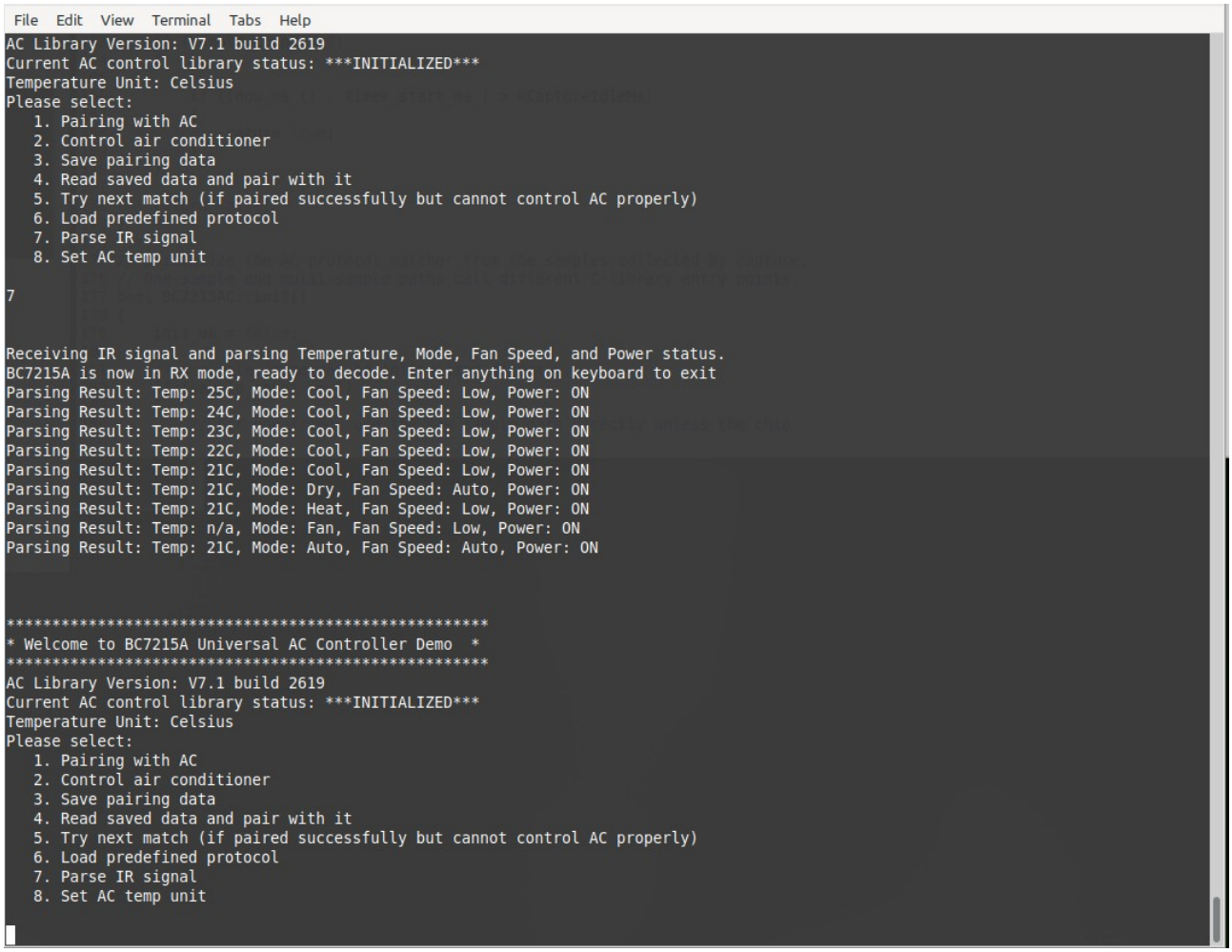
See the figure below for the user interface:



3. IR Command Parsing

After pairing succeeds, IR commands can also be parsed. Note that parsing is valid only for the paired air conditioner remote control. Select the following item in the main menu:
7 - Parse IR Signal
to enter parsing mode. After parsing mode is entered, the BC7215A switches to receiving mode. At this time, if you press a remote-control key pointing it at the IR receiver, the program parses and prints the four parameters in the command: temperature, mode, fan speed, and power status.

In parsing mode, press Enter to return to the main menu.



4. Others

Saving and reading pairing information uses the NVS of the ESP-IDF framework. However, the storage function is not part of the air conditioner library, and users may also use other methods to store pairing data.

For the purpose of the operations for finding the next match and loading a predefined protocol, refer to the BC7215A Offline Air Conditioner IR Control Library User Manual.

Using the C++ Classes

The C-language library is wrapped into C++ classes, all placed in the "bc7215" namespace.

bc7215::BC7215 Class

According to the ESP-IDF environment, this class integrates the UART operation part, so users no longer need to provide low-level UART functions. The constructor parameters are the UART number and pin assignments.

The BC7215 class provides the same functions as the C driver. Its public functions correspond to the C-language version and have the same functionality. However, the following overloaded functions are provided so that references to bc7215DataMaxPkt_t and bc7215FormatPkt_t data can be input directly, eliminating the need for cumbersome pointer type conversion:

uint8_t get_data(bc7215DataMaxPkt_t& target);

uint8_t get_format(bc7215FormatPkt_t& target);

void ir_tx(const bc7215DataMaxPkt_t& source);

bc7215::BC7215AC Class

This class wraps the C-language air conditioner code library and provides easier-to-use functions. For example, the sampling-and-pairing operation is simplified into calls to start_capture() and init(), with the specific execution logic completed inside the class. The BC7215 class is one of its members, so users do not need to define a separate BC7215 class variable during use.

Public Member Variables:

The BC7215AC class provides several externally accessible variables for convenient use:

bool init_ok; // Whether initialization (pairing) has been completed

uint8_t sample_count; // Number of sampled data segments, up to 4

uint8_t sample_status[4]; // Status of each segment

bc7215DataMaxPkt_t sample_data[4]; // Data packet of each segment

bc7215FormatPkt_t sample_format[4]; // Format packet of each segment

After sampling ends, if users need to display the contents of the received data, they can access these variables directly.

Public Member Functions:

Constructor: the parameters are the UART number and pin assignments. These parameters are passed to the internal BC7215 class member to complete the UART settings.

Initialization Function:

esp_err begin() ;

Before use, the begin() function must be called to complete the necessary initialization. If it fails, an error code is returned.

Temperature Unit Functions:

void set_fahrenheit();

void set_celsius();

bool is_celsius();

Functions for setting and querying the air conditioner temperature unit. If the unit is changed, pairing becomes invalid and pairing must be performed again.

Sampling Functions:

void start_capture(uint8_t rx_mode);

void stop_capture();

bool signal_captured();

IR signal capture commands. The parameter of the start-capture command is the RX mode after the BC7215 chip enters receiving mode. During pairing, complex mode (rx_mode = 1) must be used. During IR parsing, simple mode can be used to reduce data transmission.

The handling of multi-segment data is already included in the functions, so users do not need to care about the processing flow for multi-segment data sampling in C.

Initialization (Pairing) Functions:

bool init();

bool init(const bc725DataMaxPkt_t& data, const bc7215FormatPkt_t& format);

bool match_next();

bool init_predefined(uint8_t index);

The operation flow of the initialization (pairing) functions has already been wrapped inside the class. The pairing process is simplified to four function calls: start_capture(1) ==> check signal_captured() ==> stop_capture() ==> init().

For scenarios where saved pairing information is used for initialization at power-on, use the parameterized version:

init(const bc725DataMaxPkt_t& data, const bc7215FormatPkt_t& format);

When using predefined protocol data for initialization, a single call to init_predefined(uint8_t index) is enough. The parameter is the data index.

The function of match_next() is the same as bc7215_find_next_match() in the C-language version.

Air Conditioner Control Functions:

const bc7215DataVarPkt_t* set_to(int temp, int mode = -1, int fan = -1, int key = 2);

const bc7215DataVarPkt_t* on();

const bc7215DataVarPkt_t* off();

The wrapped C++ class integrates command data generation from the C-language library with IR data transmission, so users do not need to care about how to process the generated command data and transmit it by IR. All air conditioner control commands are condensed into these three functions. After calling these functions for power on/off or status setting, data transmission is completed automatically.

The class internally caches the last air conditioner status. When the air conditioner is turned on again after being turned off, the previous settings are restored. In parsing mode, each successfully parsed air conditioner setting is also synchronized to the internally cached air conditioner status.

* Most air conditioners do not have a dedicated power-on command. When the air conditioner is off, sending commands such as temperature settings will automatically turn it on.

IR Parsing Function:

bool parse(int& temp, int& mode, int& fan, int& power);

The parsing function is basically the same as the C-language version, but the output temperature is restored according to the air conditioner temperature unit. That is, when using Celsius, the temp range is 16-30; when using Fahrenheit, the output temp range is 60-88.

Other Functions:

uint8_t predefined_count() const; // Get the number of predefined protocols

const char* predefined_name(uint8_t index) const; // Get the predefined protocol name

bool is_busy() const; // Query whether it is busy; can be used to check whether IR command transmission is complete

const bc7215DataVarPkt_t* data_packet() const; // Get the base data packet

const bc7215FormatPkt_t* format_packet() const; // Get the base format packet

const char* lib_version() const; // Query the library version

const BC7215& driver() const; // Get the underlying driver; use this function when direct operation of the underlying driver library is required

When initialization information needs to be saved for use at the next power-on, it can be obtained through the commands for getting the base data packet and base format packet. It is not recommended to read directly from the public variables after sampling, because there may sometimes be multiple data packets. In addition, after initialization succeeds and before saving, the parsing function should not be executed, because the parsing function replaces the base data packet.